

Hunch

Your second opinion before applying.

A focused web app that helps students evaluate OJT and internship listings before they apply, send sensitive documents, or pay money.

Target user	Main action	Stack	Design
Students hunting for OJT or internships	Paste a post or upload a screenshot, then review an explainable risk score	React + TypeScript / Supabase / OpenAI / Vercel	Minimal dark analyzer-first interface

Executive brief

A grounded plan for a student-focused safety tool, not a promise of automatic fraud detection.

Hunch helps a student pause before applying. It accepts a job or internship post, identifies visible warning signals, explains the evidence, and gives the student a practical verification checklist.

PRODUCT IN ONE SENTENCE

Hunch gives students a fast, explainable second opinion on an OJT or internship listing before they apply, send sensitive documents, or pay money.

DECISIONS LOCKED FOR MVP

- Primary audience: students actively hunting for OJT and internship placements.
- Core experience: analyzer-first web app, not a general chatbot or job board.
- Primary artifact: risk score, evidence-backed warnings, and a before-applying checklist.
- Rules remain visible and testable; OpenAI supports explanation and checklist wording.
- Anonymous analysis works first; saving reports requires explicit sign-in and consent.
- Visual direction: minimal dark interface, high contrast, restrained motion, direct copy.

CONTENTS

01	02	03	04
Problem and users	Product workflow	AI and data architecture	Scope and boundaries
UI direction	Risks and validation	Build phases	Demo and decisions

Product definition and real problem

Hunch is a decision-support product for the moment before a student applies.

Students encounter OJT posts through social media, messaging apps, school groups, email, and forwarded screenshots. The posts can be incomplete, informal, or intentionally deceptive. A student may notice a fee request or an urgent message but still not know what to verify next.

THE SHARP PROBLEM STATEMENT

Students need a fast way to identify and understand warning signals in OJT and internship listings before they apply. Hunch makes those signals visible and actionable without pretending to deliver certainty.

Observed problem	Why it matters	Hunch response
Warning signs are scattered across a post.	A student may notice one signal but miss a pattern.	Group findings into categories with evidence and next actions.
Listings are often screenshots or forwarded messages.	Retyping adds friction and errors.	Support editable text extraction after screenshot upload.
A fake-or-real answer is too confident.	False positives and false negatives can both cause harm.	Show an estimate, uncertainty, and verification checklist.
Students need to choose between opportunities.	The safer-looking option may not be the most obvious one.	Save reports and compare signals side by side.

PRIMARY USER

A student actively looking for an OJT placement who may be evaluating posts from Facebook, Messenger, email, LinkedIn, school groups, or forwarded messages.

POSITIONING

Hunch is	Hunch is not
A student safety and decision-support tool	A job board or recruiting marketplace
An explainable risk estimator	A fraud verdict or legal authority
A structured checklist for safer next steps	A public company-rating platform

End-to-end product workflow

A short visible sequence: submit, understand, verify, and decide.

01	Stage	Student action	Hunch response
01	Submit	Paste a listing or upload a screenshot.	Accept the input and show what will be analyzed.
02	Review	Confirm or edit extracted text.	Make OCR uncertainty visible before analysis.
03	Analyze	Start the check.	Run rules, then request structured AI explanations.
04	Understand	Read score, evidence, and missing details.	Explain signals in plain language and show score impact.
05	Verify	Complete practical checks before applying.	Generate a checklist based on actual findings.
06	Save or compare	Save the report or compare it later.	Persist only after explicit consent and keep it private.

DETAILED WORKFLOW CONTRACT

Stage	Input	Output	Exit gate
1. Intake	Text, source label, or screenshot	Normalized text and input quality state	User can see what will be analyzed.
2. Text review	OCR text and uncertainty	User-confirmed text	Low-confidence extraction is never scored silently.
3. Rule scan	Normalized listing	Findings, categories, score impacts, evidence	Every result has a stable rule ID.
4. AI explanation	Listing plus rule findings	Summary and tailored checklist	Response validates against schema.
5. Result review	Score and findings	Risk level, evidence, next actions	Student can explain the result.
6. Persistence	Accepted report	Private saved snapshot	RLS protects user-owned data.
7. Comparison	Saved reports	Signal and score comparison	No guarantee is implied by a lower score.

CORE UX RULE

Never hide uncertainty behind a polished score. Label what was found, what was inferred, what was not provided, and what the student should verify independently.

What Hunch produces

The value is a reviewable risk report, not a mysterious number.

ANALYSIS REPORT

- Risk score from 0 to 100: Low risk 0-30, Caution 31-65, High risk 66-100.
- Plain-language summary and a clear uncertainty statement.
- Red flags grouped by category with matched evidence from the submitted post.
- Score impact, confidence, missing information, and recommended next actions.
- Before-applying checklist and analysis version/timestamp.

INITIAL DETECTION CATEGORIES

Category	Example signal	Required response
Payment request	Training, registration, processing, or security fees.	Flag strongly and recommend no payment before independent verification.
Suspicious email	Personal domain, misspelling, or domain mismatch.	Show the exact address and recommend checking the official company domain.
Vague company	Missing company name, website, or inconsistent identity.	Mark missing information and recommend independent company lookup.
Vague role	No responsibilities, supervisor, schedule, or OJT hours.	Ask the student to confirm role, hours, and reporting structure.
Unrealistic offer	Guaranteed high pay without clear duties or hours.	Flag mismatch between compensation and visible role details.
Pressure or chat-only hiring	Urgency, instant hiring, WhatsApp-only contact.	Recommend slowing down and verifying through an official channel.
Sensitive documents	IDs, bank data, passwords, or documents requested early.	Recommend withholding sensitive data until the placement is verified.

SAVED REPORT CONTRACT

A saved report is a private snapshot. It preserves submitted text, score, findings, checklist state, analysis version, and date so a later rule change does not silently rewrite history.

AI experience and architecture

Use a staged, schema-first workflow with deterministic rules as the baseline.

The app should not present several autonomous agents for show. A server-side workflow with explicit responsibilities is easier to validate, explain, and recover when a model call fails.

Stage	Responsibility	Structured result	Validation
Intake normalizer	Clean input and identify low-quality text.	NormalizedInput	Limits and required fields.
Rule engine	Detect known signals deterministically.	RuleFinding[]	Stable IDs and score bounds.
Risk interpreter	Explain the pattern without certainty claims.	RiskSummary	Evidence and tone checks.
Checklist builder	Turn findings into practical actions.	ChecklistItem[]	Specific, non-accusatory action.
Critic pass	Find unsupported claims or contradictions.	ReviewFinding[]	Critical issues block or fall back.

MINIMUM RED-FLAG OBJECT

category, title, severity, explanation, evidence, scoreImpact, confidence, nextAction, ruleId, source

FALLBACK ORDER

1. Run rules. 2. Request structured AI interpretation. 3. Validate the response. 4. If AI fails, render a useful rule-only report. 5. If rules fail, show a clear input error and do not invent a result.

ARCHITECTURE PRINCIPLE

Keep accepted analysis state as structured data. Render score, findings, checklist, and saved records from validated objects so the report cannot drift apart.

OpenAI API plan

Use AI where it creates structured judgment, while keeping secrets and safety controls on the server.

Capability	Hunch use	Boundary
Structured response	Return summary, explanations, and checklist in a known schema.	Schema validity does not guarantee factual quality.
Server-side request	Call OpenAI from a Vercel serverless route.	Never expose the API key in the browser.
Progress or streaming	Show stages only when backend events support them.	Do not fake precise progress percentages.
Image/OCR support	Support screenshot path after text extraction is stable.	Let the user review extracted text before scoring.
Safety review	Soften unsupported accusations and sensitive-data claims.	Describe signals in the post, not people as criminals.

SUGGESTED REQUEST SEQUENCE

A	B	C	D	E	F
Normalize	Rule scan	Generate	Validate	Reconcile	Render

- Set input and output length limits and reject empty input before any model call.
- Send normalized text plus rule findings, not unsupported external company claims.
- Version prompts, model choice, schemas, and saved report format together.
- Log latency, failures, and repair attempts without storing unnecessary sensitive content.
- Keep a deterministic sample result for the demo and for outage fallback.

NOT REQUIRED FOR MVP

Embeddings, a vector database, fine-tuning, realtime voice, autonomous browsing, and a public community-reporting system are deliberately deferred until the core analyzer has evidence of value.

Supabase data and security plan

Persistence adds value only when saved reports remain private and understandable.

Entity	Important fields	Purpose
profiles	id, display_name, created_at	Optional account metadata.
analyses	id, user_id, source_type, original_text, risk_score, risk_level, summary, analysis_version, created_at	Private report snapshot.
red_flags	analysis_id, rule_id, category, title, severity, evidence, score_impact, confidence	Explainable findings.
checklist_items	analysis_id, label, reason, completed, position	Report-specific actions.
saved_assets	analysis_id, storage_path, consent, created_at	Optional screenshot storage after consent.
scam_patterns	category, title, description, example, safety_tip	Curated Scam Guide content.

SECURITY REQUIREMENTS

- Enable Row Level Security on every user-owned table.
- Require `auth.uid() = user_id` for reads, writes, updates, and deletes.
- Never expose service-role credentials in the frontend.
- Use private storage access for saved screenshots and delete them with the report.
- Allow anonymous analysis without creating a saved database row.
- Give users a clear delete-data action and explain what is retained.

MVP boundaries

One credible workflow: analyze a listing, understand the signals, and decide what to verify next.

Build now	After the core flow	Explicitly out of scope
Analyzer-first home screen	Screenshot OCR	Guaranteed fraud detection
Pasted listing analysis	Saved-report comparison	Public company ratings
Rule-based score and warnings	PDF export	Automatic employer outreach
Server-side AI explanation	Company-domain helper	Full job marketplace
Auth and private saved reports	Personal pattern analytics	Legal advice or policy interpretation
Responsive dark UI and states	Moderated community reporting	Fine-tuning before a measured dataset

MVP ACCEPTANCE CHECKLIST

- A first-time student can paste a post and understand the result without prior explanation.
- Every score-changing finding has a category, evidence, explanation, and next action.
- High-risk wording never claims certainty or criminality.
- The AI route is server-side and schema-validated with a rule-only fallback.
- Anonymous analysis works without saving private content.
- A signed-in user can save, reopen, compare, and delete their own report.
- The main flow works on a narrow mobile viewport without overlap or horizontal scrolling.

SCOPE TEST

If a feature does not help a student analyze a listing, understand the evidence, choose a safer next step, or review a private decision later, it does not belong in the first release.

UI and visual direction

A minimal dark interface that puts the analyzer and the explanation in the first viewport.

Element	Direction	Guardrail
Palette	Deep neutral background, cool blue actions, green/amber/red status colors.	Status color is paired with labels and icons.
Layout	Analyzer first; result beside or below it; compact dashboard density.	Do not put a marketing page before the main action.
Typography	Legible sans-serif with restrained hierarchy.	Avoid decorative type in dense analysis content.
Shape	Thin borders, modest 8px radius, limited elevation.	Avoid nested cards and ornamental glass effects.
Motion	Short input, analysis, score reveal, and expand transitions.	Motion communicates state, never fake waiting.
Result	Score, level, summary, evidence, warnings, checklist.	Keep the score near its explanation.

FIRST-VIEWPORT CONTENT

Area	Contents
Header	Hunch wordmark, Saved, Guide, and account access.
Analyzer	Paste input, upload control, sample post, clear action, and Analyze button.
Result preview	Empty before analysis; after analysis, score and first findings appear without losing the input.
Trust note	Short statement that the result is an estimate and not proof of legitimacy.

CORE INTERACTION STATES

Empty: show a sample and an immediate input action. Loading: show a real analysis stage. Success: reveal score and evidence. Error: preserve input and offer retry or rule-only fallback. Saved: confirm private persistence. Deleted: return to a recoverable empty state.

Risks and mitigations

Hunch is promising only if it makes uncertainty more useful instead of making it look certain.

Risk or hard question	Honest answer	MVP mitigation
Is Hunch just ChatGPT with a score?	It will feel that way if every result is free-form.	Keep rules, evidence, score impacts, schemas, and checklist actions visible.
Can a low-risk result be trusted?	No. A post can omit information or use a new tactic.	Use estimate language and always show independent verification steps.
Will students overreact to false positives?	Some legitimate posts use informal channels.	Explain evidence and confidence; do not let one signal decide alone.
Can AI invent a company fact?	Yes, if prompts allow open-ended claims.	Restrict explanations to submitted evidence and missing information.
Will screenshots contain sensitive data?	They may include names, phone numbers, or IDs.	Review text before analysis; request consent before storage.
Does the score create false authority?	A number can look more certain than it is.	Show method, ranges, evidence, uncertainty, and next actions together.

NON-NEGOTIABLE BOUNDARIES

- Hunch flags signals in the submitted post; it does not declare a person or company criminal.
- A score is an estimate, not proof, legal advice, or a guarantee of safety.
- Sensitive screenshots are not stored unless the user saves and consents.
- The system must remain useful when OpenAI is unavailable.

Validation and success criteria

Measure whether students make better-informed decisions, not whether the app produces more text.

Layer	Method	Minimum evidence
Problem	Talk to 5-8 students currently seeking OJT.	At least 4 describe repeated uncertainty or exposure to suspicious posts.
Workflow	Observe first-time users with safe, suspicious, and incomplete listings.	At least 4 of 5 complete without facilitator rescue.
Detection	Run labeled fixtures through the rule engine.	Expected warnings appear with stable categories and bounded scores.
Explainability	Ask users to explain why a listing received its result.	Users can identify evidence and the recommended next step.
Reliability	Simulate AI timeout, invalid JSON, empty input, OCR failure.	Every case returns a useful fallback or recovery path.

CORE METRICS

- Time from opening the app to a readable result.
- Percentage of findings tied to visible evidence.
- Rule fixture precision and recall for the initial labeled set.
- Percentage of users who complete at least one verification step.
- Save/reopen success and delete success.
- API failure rate and median analysis latency.
- Critical accessibility issues in keyboard and mobile review.

TEST INTEGRITY

Freeze the fixture text, expected findings, and scoring rubric before evaluating a change. Keep a small golden set so improvements do not quietly trade one risk category for another.

Build phase map

Fourteen quality gates turn the product plan into an implementable solo-student roadmap.

Phase	Outcome	Key deliverables	Exit gate
1. Scope	Stable product promise and trust boundary.	Brief, fixture set, MVP contract	Scope and fixtures approved
2. Flow	Complete user journey and state coverage.	Site map, journeys, state matrix	No undefined core state
3. Research	Calibrated categories and language.	De-identified fixtures, labels	Each rule has examples
4. Wireframes	Understandable analyzer and report.	Wireframes, copy, responsive notes	Users can complete flow
5. Visual system	Cohesive dark UI and status rules.	Tokens, components, content rules	Contrast and cohesion pass
6. Foundation	Maintainable React/TypeScript base.	Routes, types, mocks, scripts	Install/build/lint pass
7. Static UI	Full mock-data journey.	Screens, states, fixtures	Demo works without backend
8. Rule engine	Transparent baseline score.	Rules, catalog, tests	Golden fixtures pass
9. OpenAI API	Evidence-bound explanation.	Route, schemas, fallback	Invalid AI cannot break UI
10. Supabase	Private saved reports.	Auth, migrations, RLS	Second user is isolated
11. OCR	Editable screenshot path.	Upload, extraction, review	No silent low-confidence scoring
12. Support tools	Comparison, Guide, checklist depth.	Saved filters, compare, content	Comparison stays cautious
13. Safety	Responsible and accessible product.	Review log, privacy, fixes	No critical safety/privacy issue
14. Ship	Live, tested, portfolio-ready demo.	QA, Vercel, README, rehearsal	Fresh reviewer succeeds

WORKING CADENCE

Complete the phase deliverables, run the exit gate, record open issues, and only then start the next phase. Keep one owner for the end-to-end demo path, even in a solo project.

Detailed phases 1-4

Define the product and its language before code creates momentum in the wrong direction.

PHASE 1: PRODUCT DEFINITION AND SCOPE LOCK

Outcome: One stable promise, target user, trust boundary, and MVP contract.

Workstreams

- Confirm the student OJT scenario and first listing sources.
- Define risk vocabulary and what evidence can affect the score.
- Write the disclaimer, privacy expectations, and non-goals.
- Create safe-looking, suspicious, incomplete, and adversarial fixtures.

Deliverables: Product brief, scope list, trust copy, fixtures, acceptance checklist.

Dependency: None.

Exit gate: The MVP fits in one minute and each rule category has a reason to exist.

PHASE 2: INFORMATION ARCHITECTURE AND USER-FLOW CONTRACT

Outcome: A complete map from opening Hunch to saving or discarding a result.

Workstreams

- Map Analyze, Result, Saved, Compare, Guide, Checklist, and Settings.
- Define anonymous and signed-in behavior plus screenshot-specific flow.
- List input, loading, partial, empty, error, retry, save, delete, and sign-out states.
- Write routes and navigation before implementation.

Deliverables: Site map, journeys, screen inventory, state matrix, route notes.

Dependency: Phase 1.

Exit gate: A student can explain each primary action and no screen depends on an undefined state.

PHASE 3: RESEARCH AND FIXTURE CALIBRATION

Outcome: Realistic student language and a defensible initial fixture set.

Workstreams

- Review examples with students seeking OJT and remove identifying information.
- Separate observed signals from assumptions about legitimacy.
- Label categories, evidence spans, confidence, and false-positive risks.

Deliverables: De-identified fixture library, category definitions, vocabulary notes, risk log.

Dependency: Phase 1.

Exit gate: Each initial rule has a positive example, a non-triggering example, and a documented rationale.

PHASE 4: UX WIREFRAMES AND INTERACTION CONTRACT

Outcome: An analyzer-first flow and understandable result report before visual polish.

Workstreams

- Wire paste, upload, sample, clear, analyze, score, evidence, warning, and checklist actions.
- Design saved report and comparison surfaces.
- Design OCR review and failure fallback.
- Check mobile, tablet, and desktop layouts.

Deliverables: Wireframes, annotated interactions, responsive rules, key-state copy deck.

Dependency: Phases 1-3.

Exit gate: Users identify the input action, risk level, and next step without narration.

Detailed phases 5-8

Make the interface real, then establish a transparent scoring baseline before connecting AI.

PHASE 5: VISUAL DESIGN SYSTEM AND CONTENT DESIGN

Outcome: A minimal dark system that keeps risk, evidence, and actions readable.

Workstreams

- Finalize palette, type, spacing, shape, borders, elevation, and status indicators.
- Write direct, non-alarmist copy and define motion/reduced-motion rules.
- Build a component inventory from the wireframes.

Deliverables: Token sheet, component variants, status rules, disclaimer rules, representative screen.

Dependency: Phase 4.

Exit gate: Analyzer and report feel like one product and core contrast/readability checks pass.

PHASE 6: FRONTEND FOUNDATION AND DEVELOPER WORKFLOW

Outcome: A maintainable React and TypeScript base ready for the product flow.

Workstreams

- Confirm Vite, routing, styling, and folder conventions.
- Create domain types and client service interfaces.
- Add lint, format, typecheck, build, environment examples, and mock services.

Deliverables: App shell, routes, types, mocks, scripts, README setup notes.

Dependency: Phases 1-5.

Exit gate: A new contributor can install, run, typecheck, lint, and build from the README.

PHASE 7: STATIC UI AND INTERACTION BUILD

Outcome: The complete journey works with controlled mock data.

Workstreams

- Build analyzer, result, checklist, Saved, Compare, Guide, and Settings views.
- Implement responsive behavior, keyboard focus, loading, empty, error, retry, and no-result states.
- Add safe, caution, high-risk, and incomplete mock fixtures.

Deliverables: Clickable frontend, mock fixtures, responsive states, visual screenshots.

Dependency: Phase 6.

Exit gate: The full journey demos without backend services and no core state is only a happy-path screen.

PHASE 8: RULE-BASED DETECTION ENGINE

Outcome: A transparent, testable baseline for risk scoring.

Workstreams

- Normalize text and implement fee, email, company, role, compensation, urgency, chat-only, and sensitive-document rules.
- Define severity, score impact, evidence spans, deduplication, and 0-100 normalization.
- Record rule version and add unit/fixture tests.

Deliverables: Rule module, catalog, score breakdown, fixture suite, deterministic reports.

Dependency: Phases 1, 3, 6, and 7.

Exit gate: Every score-changing result is explainable and golden fixtures trigger expected warnings.

Detailed phases 9-11

Add AI and persistence only after the deterministic product loop is understandable.

PHASE 9: OPENAI ANALYSIS API

Outcome: AI adds evidence-bound explanation without becoming the score's source of truth.

Workstreams

- Create a secure Vercel route with request and response schemas.
- Send normalized text plus rule findings and generate summary, uncertainty, and checklist items.
- Validate output and provide timeout, rate, length, repair, and rule-only fallback behavior.
- Log non-sensitive operational metadata and version prompts.

Deliverables: Server-side service, prompts, schemas, mapper, fallback path, failure fixtures.

Dependency: Phases 6-8 and current official API documentation.

Exit gate: Keys stay server-side, invalid AI cannot break the UI, and explanations cite available evidence only.

PHASE 10: SUPABASE AUTHENTICATION AND PERSISTENCE

Outcome: Students can privately save and revisit report snapshots.

Workstreams

- Create Auth, migrations, indexes, foreign keys, and RLS policies.
- Save report snapshots, checklist state, and analysis version.
- Test multiple-user isolation and add delete-report and asset deletion.

Deliverables: Migrations, auth screens, save/open/delete services, RLS tests, privacy copy.

Dependency: Phases 6-9.

Exit gate: Anonymous users analyze; signed-in users save/reopen; a second user cannot access the first report.

PHASE 11: SCREENSHOT OCR AND INPUT QUALITY

Outcome: Students can use screenshots without trusting unreviewed extraction.

Workstreams

- Add upload constraints and privacy guidance.
- Extract text, mark uncertainty, and let users edit before analysis.
- Prevent storage without explicit save and consent.
- Test low-resolution, cropped, multilingual, and text-heavy images.

Deliverables: Upload flow, extraction review, OCR fallback, optional storage path, input-quality set.

Dependency: Phases 7, 8, and 10.

Exit gate: OCR never silently scores missing text and failed uploads return cleanly to paste input.

Detailed phases 12-14

Extend the useful decision workflow, then harden and ship it as portfolio evidence.

PHASE 12: COMPARISON, SCAM GUIDE, AND CHECKLIST DEPTH

Outcome: Hunch supports the decision after the first score without becoming a job board.

Workstreams

- Add saved filters and sorting by risk, source, and date.
- Build side-by-side comparison of score, findings, missing information, and checks.
- Write curated Guide content from the rule catalog and reusable checklist templates.

Deliverables: Saved list/detail views, comparison view, Guide entries, checklist templates.

Dependency: Phases 8 and 10.

Exit gate: A student can compare listings without treating a lower score as a guarantee of safety.

PHASE 13: SAFETY, PRIVACY, ACCESSIBILITY, AND ABUSE REVIEW

Outcome: The product handles uncertainty and sensitive content responsibly.

Workstreams

- Review claims, prompts, screenshot retention, deletion, sign-out, and error recovery.
- Add rate limits and anonymous-analysis abuse controls.
- Test keyboard flow, focus, contrast, labels, reduced motion, and mobile wrapping.

Deliverables: Safety log, privacy copy, accessibility fixes, abuse notes, final disclaimer.

Dependency: Phases 9-12.

Exit gate: No critical safety, privacy, unsupported-claim, keyboard, or contrast issue remains.

PHASE 14: TESTING, DEPLOYMENT, AND PORTFOLIO PROOF

Outcome: A stable live project that can be evaluated in a short demo.

Workstreams

- Run unit, integration, end-to-end, manual fixture, fresh-browser, auth-boundary, and failure tests.
- Configure Vercel, Supabase production settings, secrets, redirects, and smoke tests.
- Write README, architecture notes, screenshots, demo script, limitations, and deterministic fallback.

Deliverables: QA checklist, production deployment, environment notes, README, screenshots, rehearsal.

Dependency: All prior phases.

Exit gate: A fresh reviewer analyzes a fixture, understands the result, reviews the checklist, and sees the technical boundaries within five minutes.

SOLO PROJECT ROLES

Role	Owns	Focus
Product/research	Scope, student interviews, fixtures, acceptance criteria	Keep the problem narrow and grounded.
Frontend/design	Flow, dark system, responsive states, content	Make the analyzer immediate and trustworthy.
AI/backend	Rules, prompts, schemas, API, fallback	Make analysis explainable and observable.
Data/security	Supabase schema, RLS, storage, deletion	Protect private reports and screenshots.
QA/demo	Fixtures, failures, screenshots, README, rehearsal	Prove the product beyond the happy path.

Demo story and decision log

The strongest portfolio story proves that Hunch changes a student's next action.

THREE-MINUTE DEMO

Time	Action	What the audience learns
0:00-0:20	Open Hunch and show the analyzer immediately.	The product starts with the student's real task.
0:20-0:45	Paste a suspicious OJT post with a fee request and vague role.	Hunch accepts realistic, messy input.
0:45-1:15	Run analysis and reveal the score.	The score is only the beginning of the explanation.
1:15-1:55	Expand warnings to show evidence and score impact.	The result is traceable instead of a black-box label.
1:55-2:25	Complete verification checklist items.	Hunch turns concern into safer next actions.
2:25-2:45	Save and compare with a lower-risk listing.	The product supports a real decision.
2:45-3:00	Show architecture and limitations.	The project uses AI carefully and honestly.

PORTFOLIO PROOF POINTS

- A staged AI workflow with structured output and fallback behavior.
- A deterministic detection layer that makes the score explainable.
- Supabase Auth, Postgres, RLS, and optional private storage.
- A responsive analyzer-first interface with explicit state design.
- Safety, privacy, accessibility, and uncertainty treated as product requirements.

DECISION LOG

Decision	Reason
React + TypeScript	Typed, interactive frontend and clear portfolio demonstration.
Supabase	Auth, Postgres, RLS, and storage without a custom backend from scratch.
OpenAI through a server route	Structured explanation while keeping secrets and control on the server.
Rule engine before AI	Transparent baseline prevents the score from depending on prose generation.
Paste text before OCR	Proves the core loop before adding image-extraction complexity.
Private saved reports	History value without creating a public accusation system.

FINAL SCOPE TEST

If a feature does not help a student analyze a listing, understand the evidence, choose a safer next step, or review a private decision later, it should not be part of the first Hunch release.